AI / ML Engineer Roadmap

A Revised, Honest Path from Beginner to Hireable

15 Months • 6 Phases • Project-First Learning

Built on the strengths of the original roadmap, with added modules for SQL, cloud fundamentals, LLM fine-tuning, and application development — the gaps that actually show up in hiring loops today.

How This Roadmap Is Different

Read this before starting Phase 1

What Stays the Same

The original roadmap you followed got many things right: the Python-first foundation, the refusal to jump into deep learning before classical ML, the insistence on building from scratch before reaching for sklearn, and the heavy weighting on MLOps. All of that is preserved here. Good engineering habits beat clever shortcuts every time.

What Changes

Three gaps are closed. **SQL** is introduced in Phase 1 because almost every data and ML role screens for it. **Cloud fundamentals** (one provider, not three) are added to Phase 4 because deploying only to localhost is not a deployable skill. **LLM application development** — RAG, embeddings, evaluation — is added as a dedicated module between Deep Learning and MLOps, because junior roles increasingly expect it and because understanding the fundamentals first (as you will) is exactly what makes this module land properly.

The timeline is also corrected. The original promised 52 weeks. This one budgets 62 weeks (roughly 15 months) with explicit buffer weeks. Backpropagation will take longer than planned. So will your first real deployment. Planning for that honestly is better than pretending it won't happen.

Core Principles

- **Learn → Build → Explain → Repeat.** Every concept ends with working code and a written explanation.
- **One source per topic.** Do not collect courses. Finish one, move on.
- **Ship in public.** GitHub push per week, LinkedIn post per two weeks minimum.
- **Buffer weeks are real weeks.** They are not vacation. They are for consolidation, debugging, and revisiting what did not click.
- **Projects over topics.** If you can build it and explain it, you know it. If you cannot, you do not.

> ***A note on expectations.*** *At the end of 14 months you will not be an AI expert. You will be a competent entry-level AI/ML engineer with a real portfolio, a deployed system, and the ability to discuss tradeoffs in an interview without bluffing. That is the honest target. Anyone promising more in this timeframe is selling something.*

Phase Map at a Glance

Phase	Weeks	Focus	Outcome
1	1–6	Foundations	Python, NumPy, Pandas, SQL, math intuition
2	7–22	Classical ML	Regression, trees, ensembles, feature engineering

3	23–42	Deep Learning	ANN, CNN, RNN, Transformers, one strong project
4	43–50	LLM Apps + Fine-Tuning	RAG, embeddings, LoRA, LLM eval, one deployed app
5	51–58	MLOps + Cloud	APIs, Docker, CI/CD, monitoring, one cloud deploy
6	59–62	Capstone + Interview	End-to-end system, mock interviews, resume

In Parallel Throughout All Phases

- **GitHub:** One repository per project. Clean README with problem, approach, and results.
- **LinkedIn:** One post every two weeks. Share what broke and how you fixed it — that reads more credible than success-only posts.
- **LeetCode:** 2–3 problems per week. Arrays, strings, hash maps. Not a DSA grind.
- **Reading:** One ML blog post or paper summary per week. Gradient, The Batch, or Sebastian Raschka's newsletter are good anchors.
- **Community:** Join one Discord or Slack (MLOps Community, Hugging Face, or a local AI meetup) by month 3.

What This Roadmap Deliberately De-emphasizes

Other roadmaps push these topics hard. This one does not — for specific reasons. You should know what these are and why they are light here, so you can decide for yourself:

- **Reinforcement Learning:** Research-heavy, rarely asked in junior interviews, easy to spend 2 months on for zero hiring benefit. Covered conceptually only, enough to answer 'what is RL?' in an interview.
- **Kubernetes:** A second-job skill. Deploying to ECS Fargate, App Runner, or Cloud Run is enough for junior roles. Kubernetes becomes relevant once you have scale you do not yet have.
- **Distributed training:** You will not train a model that needs multi-GPU in your first role. Conceptual awareness only.
- **GANs:** Famous but rarely used in entry-level work. Covered briefly for completeness.
- **Heroku:** The free tier is gone. Use cloud providers directly instead.
- **Every framework:** Pick PyTorch over TensorFlow. Pick one LLM orchestration library, not three. Breadth without depth is the tutorial trap.

If you finish this roadmap and decide RL or Kubernetes or distributed training are what you want next — great. Learn them then, with the foundation to absorb them properly. Just do not front-load them.

Phase 1 — Foundations

Weeks 1–6 • Python, Data, SQL, Math Intuition

Goal

Build a foundation so solid that the rest of the roadmap feels like applying what you already know, not learning from scratch. By the end of Phase 1 you should think in Python, think in arrays, think in tables, and query a database without flinching.

Rules

- No machine learning algorithms yet — resist the urge.
- Every concept ends with working code and a short written explanation of why it works.
- Every week: core learning + one mini project + one GitHub push.
- Errors are not failures. Reading stack traces is a core skill being built here.

Week 1 — Python Core

Topics

- Variables as references, not containers
- Data types, type casting, input/output
- Conditions, loops, break/continue/pass
- Functions: parameters, return vs print, scope

Key Insight

Python variables point to objects. Understanding this prevents half of all future bugs involving mutable defaults, list aliasing, and unexpected side effects.

Practice

- CLI calculator with four operations
- Temperature and currency converters
- Pattern printing (at least 5 patterns)
- Number guessing game with difficulty levels

Week 2 — Data Structures & File Handling

Topics

- Lists: indexing, slicing, methods, mutability
- Tuples, sets, dictionaries, nested dictionaries
- String methods and f-strings
- File I/O: read, write, CSV basics
- List and dictionary comprehensions

Mini Project: Student Result Analyzer

- Read student data from CSV
- Compute percentage, assign grades
- Write summary report to file
- Handle missing/malformed rows gracefully

Deliverable

GitHub repo with README explaining the problem, logic, and file structure.

Week 3 — Python Advanced — OOP & Debugging

Topics

- Exception handling: try, except, finally, custom exceptions
- Modules, packages, imports, virtual environments, pip
- Classes, __init__, instance methods, inheritance, encapsulation
- Reading stack traces, using print-debugging and pdb
- Naming conventions, PEP 8, type hints (introductory)

Mini Project: Expense Tracker (CLI, OOP)

- Expense class, Category class, Tracker class
- Add, categorize, summarize, persist to JSON
- Exception handling for bad inputs

Why this matters

Every serious ML library — PyTorch, scikit-learn, Hugging Face — is class-based. If OOP feels foreign, those libraries will feel like magic. They are not magic.

Week 4 — NumPy — Array Thinking

Topics

- ndarray creation, shape, dtype, indexing, slicing
- Broadcasting rules (spend real time here)
- Vectorized operations vs Python loops
- Axis, aggregation, reshape, view vs copy
- Dot product, norms, basic linear algebra

Mini Project: Stats Engine

- Load a real CSV without pandas
- Compute mean, std, min, max, percentiles using NumPy
- Normalize columns (z-score and min-max)
- Plot distributions with matplotlib
- Benchmark vectorized vs loop-based versions — show the speedup

Deliverable

Notebook + README with a benchmark table showing NumPy is 50–500x faster than pure Python loops on your data.

Week 5 — Pandas & Real Data

Topics

- Series vs DataFrame, loc vs iloc
- Filtering, sorting, groupby, apply
- Merging, joining, concatenation
- Missing values: detect, drop, impute
- Datetime handling and basic time-series

Mini Project: Real Dataset EDA

- Pick a Kaggle dataset (sales, finance, or health)
- Data cleaning notebook with every decision documented
- At least 5 charts answering specific questions
- Markdown summary of 3 non-obvious findings

Key mindset

A messy real dataset will teach you more in one week than ten tutorials on clean data.

Week 6 — SQL & Math Intuition (new — closes a major gap)

SQL Topics

- SELECT, WHERE, ORDER BY, LIMIT
- JOINs: inner, left, right, full
- GROUP BY, HAVING, aggregate functions
- Subqueries and CTEs (WITH clause)
- Window functions: ROW_NUMBER, RANK, running totals

Math Topics (intuition, not proofs)

- Vectors, matrices, matrix multiplication — what it does geometrically
- Eigenvalues and eigenvectors — the 30-second version, because PCA needs it
- Mean, variance, standard deviation — when each matters
- Probability basics: independence, conditional probability, Bayes theorem
- Probability distributions: Normal, Binomial, Poisson — and when you see each
- Derivatives as slope; partial derivatives; why gradient descent works

Practice

- Solve 30 SQL problems on StrataScratch or LeetCode SQL (easy + medium)
- Khan Academy: Linear Algebra unit 1, Statistics unit 1
- Write a short post: 'Why vectorization works — a geometric view'

Phase 1 Capstone

Combine it all: take a Kaggle dataset, load it into SQLite, write 10 SQL queries that answer business questions, then do the same analysis in pandas. Compare. Publish as one polished repo.

> ***End of Phase 1 checklist.*** *You should have 6 GitHub repos, one capstone project, comfort with Python + NumPy + Pandas + SQL, and at least 3 LinkedIn posts. If any of these are missing, do not proceed. Phase 2 assumes all of it.*

Phase 2 — Classical Machine Learning

Weeks 7–22 • The phase that actually gets you hired

The Hard Truth

Around 70–80% of real junior ML interviews focus on classical ML: regression, trees, ensembles, evaluation, feature engineering, and problem framing. Deep learning is interview-flashy but classical ML is interview-decisive. Give this phase the weight it deserves. Do not rush it.

Learning Loop (applied to every algorithm)

- **Intuition first** — why does this algorithm exist? What problem does it solve?
- **Only the math you need** — enough to implement, not enough to write a paper.
- **From-scratch implementation** — NumPy only, no sklearn.
- **sklearn on a real dataset** — compare your scratch version, tune, evaluate.

Weeks 7–8 — ML Foundations

Topics

- What ML actually is (and isn't)
- Supervised, unsupervised, semi-supervised
- Full workflow: framing, data, features, training, evaluation
- Train / validation / test split; why leakage is the #1 beginner mistake
- Overfitting, underfitting, bias–variance tradeoff
- Metrics: accuracy, precision, recall, F1, ROC-AUC, confusion matrix
- When ML should NOT be used — this is an interview favorite

Mini Project: Problem Framing Case Study

Pick three real problems (loan approval, house prices, spam detection). For each: define features, target, metric, risks, and at least one failure mode. Deliverable is a markdown document — no code yet.

Weeks 9–10 — Linear Regression — The Backbone

Topics

- Simple vs multiple linear regression
- Cost function (MSE) and its geometry
- Gradient descent: intuition, math, learning rate effects
- Normal equation — when it works, when it doesn't
- Feature scaling and why it matters for gradient descent
- Regularization: Ridge (L2), Lasso (L1), Elastic Net

From Scratch (required)

- Linear regression with gradient descent in NumPy
- Plot loss curve and visualize descent
- Compare convergence across learning rates

Project: House Price Prediction

From-scratch model + sklearn model + Ridge + Lasso. Compare MAE, RMSE, R^2. Error analysis: where does each model fail? Publish repo + LinkedIn post.

Weeks 11–12 — Logistic Regression — Classification Begins

Topics

- Sigmoid function and decision boundary
- Log loss (cross-entropy) and why MSE fails here
- Gradient descent for classification
- Threshold tuning and business cost of errors
- ROC–AUC and Precision–Recall curves
- Class imbalance: resampling, class weights, threshold shifts

Project: Spam Classifier

UCI or Kaggle spam dataset. Feature extraction (bag-of-words, TF-IDF). From-scratch logistic regression. sklearn comparison. Evaluate with precision–recall, not just accuracy.

Weeks 13–14 — KNN and Naive Bayes

KNN Topics

- Distance metrics: Euclidean, Manhattan, cosine
- Choosing K; the curse of dimensionality
- Why KNN is slow at inference and how to speed it up

Naive Bayes Topics

- Bayes theorem and conditional probability
- The naive independence assumption — why it works anyway
- Gaussian, Multinomial, and Bernoulli NB

Project: News Category Classification

Compare KNN vs Naive Bayes vs Logistic Regression on the same text data. Explain in writing why the winner won.

Weeks 15–16 — Decision Trees

Topics

- Gini impurity vs entropy; information gain
- How trees split and when they stop
- Overfitting in trees; max_depth, min_samples_leaf
- Pruning (conceptual)
- Feature importance and its limitations

Project: Customer Churn

Business framing matters here. What does a false negative cost? Visualize the tree. Explain top features to a non-technical reader.

Weeks 17–18 — Random Forest & Bagging

Topics

- Why ensembles beat single models (variance reduction)
- Bagging and feature randomness
- Out-of-bag error as free validation
- Hyperparameter tuning: n_estimators, max_depth, max_features

Project: Credit Risk

Compare logistic regression, decision tree, and random forest. Business-aware metric (e.g., cost-weighted misclassification). Write up tradeoffs: accuracy vs interpretability vs training time.

Weeks 19–20 — Gradient Boosting — XGBoost & LightGBM

Topics

- Boosting intuition: weak learners correcting each other
- Learning rate vs number of estimators (tradeoff)
- Overfitting control: early stopping, regularization
- Why boosting wins most tabular competitions

Project: Fraud Detection

Highly imbalanced dataset. SMOTE or class weights. XGBoost vs LightGBM. Precision–recall tradeoff is the whole story here — no accuracy bragging.

Weeks 21–22 — Unsupervised Learning, Feature Engineering, Pipelines & Capstone

Unsupervised Learning Topics

- K-Means clustering and the elbow method for choosing K
- Hierarchical clustering (agglomerative) and dendrograms
- DBSCAN — when density matters more than distance
- Principal Component Analysis (PCA) for dimensionality reduction
- When to use unsupervised methods: EDA, segmentation, anomaly detection

Feature Engineering & Pipelines

- Encoding categorical variables: one-hot, target, ordinal
- Scaling: StandardScaler, MinMaxScaler, RobustScaler
- Feature selection: filter, wrapper, embedded methods
- sklearn Pipelines and ColumnTransformer
- Cross-validation strategies (K-fold, stratified, time-series)
- GridSearchCV and RandomizedSearchCV

Mini exercise

Customer segmentation on a retail dataset using K-Means + PCA for visualization. This doubles as a common interview case study.

Phase 2 Capstone: End-to-End Classical ML System

Pick a real domain (finance, healthcare, education, marketing). Clean data, engineer features, try 3+ models, tune hyperparameters, evaluate with appropriate metrics, write a final recommendation. One polished repo with architecture diagram and README.

> ***End of Phase 2 checklist.*** *10+ ML projects on GitHub, strong intuition for which model to reach for when, the ability to explain bias–variance tradeoff in plain English, and one capstone that looks like real work. This is the phase that differentiates you from the 'I did a Kaggle tutorial' crowd.*

Phase 3 — Deep Learning

Weeks 23–42 • Representation learning, done properly

Mindset Shift

Classical ML is about engineering features. Deep learning is about letting the model learn features from raw data. This is not better or worse — it is different, and it requires different habits: more data, more compute awareness, and more patience with training curves.

Spiral Learning

You will revisit concepts. Backpropagation in week 25 will not fully land until you debug a real model in week 35. That is normal. Do not expect linear progress.

Sub-phases

Sub-phase	Weeks	Focus
3A	23–26	Neural Network Fundamentals (ANN)
3B	27–31	Convolutional Neural Networks (Vision)
3C	32–35	Sequence Models (RNN, LSTM, GRU)
3D	36–38	Autoencoders & GANs
3E	39–42	Transformers + Capstone

Weeks 23–24 — Neural Network Basics

Topics

- Why linear models fail on complex data
- Perceptron and multi-layer perceptron
- Activations: sigmoid, tanh, ReLU, leaky ReLU — and why ReLU changed everything
- Loss functions: MSE for regression, cross-entropy for classification
- Manual forward pass on a tiny 2-layer network with a pen and paper

Week 25 — Backpropagation (Critical Week)

Topics

- Chain rule — intuition first, then mechanics
- Gradient flow through layers
- Vanishing and exploding gradients
- Weight initialization: Xavier, He

From Scratch (required)

Two-layer neural network in pure NumPy. Manual forward and backward pass. Train on MNIST. Do not move on until this works.

Warning

This is the week most learners secretly skip. Do not. If it takes 10 days instead of 7, take 10 days. Everything in Phase 3 rests on this.

Week 26 — Training Neural Networks & PyTorch

Topics

- Optimizers: SGD, momentum, Adam — when each matters
- Learning rate scheduling
- Regularization: L2, dropout, batch normalization
- PyTorch essentials: tensors, autograd, nn.Module, training loop

Project: MNIST Digit Classifier

Build it twice: once from scratch in NumPy, once in PyTorch. Compare accuracy, training time, and code clarity.

Weeks 27–29 — Convolutional Neural Networks

Topics

- Why ANN fails on images (parameter explosion, no spatial awareness)
- Convolution, kernels, stride, padding, pooling
- Receptive field and parameter sharing
- Classic architectures (conceptual): LeNet, AlexNet, VGG, ResNet (skip connections)
- Data augmentation, transfer learning, fine-tuning, layer freezing

Weeks 30–31 — CNN Project

Options

- Plant disease detection (agriculture angle, uncommon on resumes)
- Face mask detection (CV pipeline practice)
- Product defect detection (industrial framing)

Requirements

- Custom CNN trained from scratch (accept lower accuracy)
- Transfer learning with a pretrained ResNet (show the improvement)
- Grad-CAM or similar visualization showing what the model looks at
- Honest failure analysis — what does it still get wrong?

Weeks 32–33 — Sequence Models: RNN, LSTM, GRU

Topics

- Sequential data and hidden state
- RNN unrolling and vanishing gradient problem
- LSTM gates (forget, input, output) and why they help
- GRU as simpler alternative
- Text preprocessing: tokenization, padding, embeddings

Weeks 34–35 — Sequence Model Project

Options

- Sentiment analysis on real reviews
- Time-series forecasting (stock, weather, or energy demand)
- Next-word prediction on a small corpus

Requirements

Clean preprocessing pipeline, LSTM baseline, honest evaluation. Explicitly document what your model cannot do — this is how you develop taste.

Weeks 36–38 — Autoencoders & GANs

Autoencoder Topics

- Encoder–decoder architecture
- Dimensionality reduction (vs PCA)
- Denoising autoencoders
- Anomaly detection use case

GAN Topics

- Generator vs discriminator — the minimax game
- Training instability and mode collapse
- Why GANs are hard (set expectations honestly)

Project

Simple GAN on MNIST or Fashion-MNIST. Document the pain of training it.

Weeks 39–40 — Attention & Transformers

Topics

- Why RNNs fail at long sequences
- Attention mechanism: query, key, value — with small numerical examples
- Self-attention and multi-head attention
- Positional encoding
- Encoder–decoder structure
- BERT vs GPT intuition (bidirectional vs autoregressive)

Reading

Illustrated Transformer (Jay Alammar) + the original 'Attention Is All You Need' paper (skim for intuition, not proofs).

Weeks 41–42 — Phase 3 Capstone

Project Options

- Image captioning (CNN + Transformer)
- Fine-tune a pretrained BERT for domain-specific classification
- Multi-modal retrieval (image + text embeddings)

Required Deliverables

- Clean, readable codebase (no single 800-line notebook)
- Training pipeline with logged metrics
- Evaluation strategy with appropriate metrics
- README with architecture diagram
- One LinkedIn post explaining what you built and what broke

Reinforcement Learning — Conceptual Sidebar (2–3 days within Phase 3)

RL is not a focus of this roadmap and you will not build an RL project. But you should be able to answer basic questions about it in interviews without sounding lost. Budget 2–3 days at any point during Phase 3 for this — no project required.

- Core vocabulary: agent, environment, state, action, reward, policy
- Exploration vs exploitation; epsilon-greedy
- Q-learning intuition (no implementation required)
- Deep Q-Networks: what they are, why they matter (AlphaGo, game-playing)
- When RL is the right tool (rare) vs when supervised learning is (usually)
- One resource: David Silver's RL lecture 1 on YouTube, or the RL chapter in Sutton & Barto (read intro only)

Phase 4 — LLM Application Development & Fine-Tuning

Weeks 43–50 • Build with LLMs, learn when and how to train them

Why This Phase Exists

The original roadmap was cautious about generative AI, and that caution was correct for beginners — understanding transformers before using them matters. But you already understand transformers now (Phase 3E). At this point, LLM application skills are not optional for junior AI/ML roles. The job market has shifted. Retrieval-augmented generation, embeddings, and LLM evaluation appear on most 2026 job descriptions.

What You Will NOT Do

- Train a foundation model from scratch (impossible and irrelevant at your stage)
- Become a prompt-engineering guru (shallow skill, short half-life)
- Chase every new framework (LangChain vs LlamaIndex vs direct API — pick one, ship)

What You Will Do

- Build a real RAG system end-to-end
- Understand embeddings, vector databases, and retrieval quality
- Fine-tune open-source LLMs using parameter-efficient methods (LoRA/QLoRA)
- Know when to fine-tune vs when RAG alone is enough — this is the real skill
- Evaluate LLM outputs systematically, not by vibes
- Ship one deployed LLM app that does something useful

Week 43 — LLM Fundamentals for Builders

Topics

- Tokenization, context windows, token costs
- Temperature, top-p, top-k — and when to tune them
- Structured output (JSON mode, function calling)
- Why you need system prompts, not just user prompts
- API patterns: Anthropic, OpenAI, open-source via Ollama

Practice

Build a simple CLI chatbot against one API. No framework. Understand the raw request/response.

Week 44 — Embeddings & Vector Search

Topics

- What embeddings are and why they work
- Cosine similarity, dot product, Euclidean — and when each matters
- Vector databases: Chroma (start here), Qdrant, Pinecone
- Chunking strategies: fixed-size, semantic, recursive
- Retrieval quality metrics (recall@k, MRR)

Project

Build a semantic search engine over your own notes or a public dataset. Evaluate retrieval quality with a small labeled test set.

Weeks 45–46 — Retrieval-Augmented Generation (RAG)

Topics

- Why pure LLMs hallucinate and why retrieval helps
- Full RAG pipeline: ingest → chunk → embed → store → retrieve → generate
- Hybrid search (keyword + semantic)
- Re-ranking with cross-encoders
- Citation and source attribution
- Common failure modes: bad chunks, retrieval misses, context overflow

Project: Domain-Specific Q&A; System

Pick a domain you care about (legal docs, research papers, your company's docs). Build a RAG app. Evaluate with at least 20 hand-crafted Q&A; pairs. Publish the evaluation results — honest numbers, not cherry-picked demos.

Week 47 — LLM Evaluation (Underrated Skill)

Topics

- Why 'it looks good' is not evaluation
- Reference-based metrics: BLEU, ROUGE — and their limits
- LLM-as-judge: setup, calibration, common biases
- Human evaluation protocols for small-scale projects
- Regression testing: why your app will silently degrade

Practice

Build an evaluation harness for the RAG system from week 45–46. Run it on every change. This is the habit that separates builders from tinkerers.

Week 48 — Agents & Tool Use (Introductory)

Topics

- Function calling / tool use — the real primitive
- When an agent makes sense (rarely) vs when a pipeline makes sense (usually)
- ReAct pattern and its limitations
- Cost and latency realities of agent loops

Mini Project

Add a tool-using capability to your RAG app — e.g., a calculator tool, or a web search tool. Keep it small. The goal is to understand the pattern, not to build AutoGPT.

Week 49 — Fine-Tuning — When, Why & How

The Decision Framework (this is what interviewers actually test)

- Prompt engineering alone: enough for ~60% of use cases. Start here. Always.
- RAG: when the model needs external or private knowledge. You built this in weeks 45–46.
- Fine-tuning: when you need to change HOW the model responds (tone, format, domain reasoning) — not just WHAT it knows.
- Full training: when you need a new model. You will not do this as a junior. Period.

Topics

- What fine-tuning actually does: updating weights on a domain-specific dataset
- Supervised fine-tuning (SFT) with instruction-response pairs
- Training data preparation: formatting, quality filtering, deduplication — garbage in, garbage out applies here harder than anywhere in ML
- Base models vs instruction-tuned models — know the difference
- Overfitting in fine-tuning: it happens fast with small datasets
- Evaluation: held-out test set, task-specific metrics, human preference

Interview-ready mental model

Prompt engineering is free and instant. RAG adds knowledge. Fine-tuning changes behavior. Full training creates new capabilities. Always try the cheaper thing first. Interviewers love candidates who can articulate this hierarchy.

Week 50 — Parameter-Efficient Fine-Tuning (LoRA/QLoRA) — Hands-On

Topics

- Why full fine-tuning is expensive: GPU memory, training time, storage of full model copies
- LoRA (Low-Rank Adaptation): freeze base model, train small adapter matrices
- QLoRA: quantize the base model + LoRA — fine-tune a 7B model on a single consumer GPU
- Hugging Face PEFT library and TRL (Transformer Reinforcement Learning) library
- Adapter merging: combining LoRA weights back into the base model for deployment
- RLHF and DPO — conceptual awareness only (what they are, not how to implement)

Project: Fine-Tune an Open-Source LLM

- Pick a small model (Mistral 7B, Llama 3 8B, or Phi-3 via Ollama/Hugging Face)
- Curate a dataset of 500–1000 instruction-response pairs for a specific domain
- Fine-tune with QLoRA using Hugging Face + PEFT
- Evaluate: compare base model vs fine-tuned on 20+ test examples
- Document the cost: how long, how much GPU, what went wrong
- If no GPU available: use Google Colab free tier or Lambda Labs spot instances

What NOT to do

- Do not fine-tune GPT-4 or Claude via their APIs and call it fine-tuning experience — that is API usage, not training
- Do not skip the data curation step. The dataset quality IS the project.
- Do not spend 3 weeks chasing SOTA. A working fine-tune with honest evaluation beats a half-finished experiment with the latest model.

> ***End of Phase 4 checklist.*** *One deployed RAG application with an evaluation harness, one fine-tuned open-source LLM with documented evaluation results, one LinkedIn post that shows you can talk about LLMs without the usual hype vocabulary, and the ability to answer 'when would you fine-tune vs use RAG?' — this question appears in nearly every 2026 AI interview. This phase alone gets attention from hiring managers.*

Phase 5 — MLOps + Cloud Fundamentals

Weeks 51–58 • From notebook to production

The Hiring Truth

Most candidates can train a model. Far fewer can deploy one. The candidates who can deploy, monitor, and maintain models get offers. This phase is where that happens.

Weeks 51–52 — ML System Thinking & Experiment Tracking

Topics

- Why notebooks fail in production (reproducibility, versioning, automation)
- Standard ML project structure (cookiecutter-style)
- Config-driven training with Hydra or simple YAML
- MLflow: tracking, model registry, artifacts
- Git practices for ML: what to commit, what to ignore, Git LFS

Project

Refactor one Phase 2 or Phase 3 project into a proper ML project structure. Track 5+ experiments in MLflow. Select and register a best model.

Weeks 53–54 — FastAPI & Model Serving

Topics

- REST API fundamentals: methods, status codes, content types
- FastAPI: path operations, Pydantic models, dependency injection
- Request validation and error handling
- Async vs sync for ML workloads (usually sync, explain why)
- Model loading patterns: load once at startup, not per request
- API documentation with Swagger (automatic in FastAPI)

Project

Serve a Phase 2 model and your RAG app from Phase 4 as FastAPI services. Test with Postman or httpie. Document with OpenAPI.

Weeks 55–56 — Docker & CI/CD

Topics

- Docker concepts: images, containers, layers, volumes
- Writing a clean Dockerfile for Python ML apps
- docker-compose for multi-service setups
- GitHub Actions: workflows, jobs, steps
- CI pipeline: lint → test → build image → push to registry
- CD considerations (we will keep this light)

Project

Dockerize both APIs from weeks 51–52. Build a GitHub Actions pipeline that runs tests and builds the image on every push. Push to Docker Hub or GHCR.

Week 57 — Cloud Deployment (new — closes the second major gap)

Choose ONE cloud

- AWS if you want the most jobs (widest adoption)
- GCP if you want the best ML tooling
- Azure if enterprise roles are your target

Topics (AWS-flavored; adapt to your choice)

- IAM: users, roles, policies — and why it matters for security
- S3 for data and model artifacts
- ECR for container images
- ECS Fargate or App Runner for simple container deployment (skip Kubernetes for now)
- **Serverless alternative:** AWS Lambda + API Gateway for lightweight models (know this pattern exists — cold starts matter, model size limits matter)
- CloudWatch logs and basic alarms
- Budget alerts — seriously, set these first

Scaling & Scaling-Out — Conceptual Awareness Only

- Load balancing and auto-scaling — know what they are, not how to configure them yet
- Distributed / multi-GPU training: understand data parallelism vs model parallelism at a vocabulary level
- Kubernetes exists and runs most large ML platforms — know that, move on

Project

Deploy your Dockerized ML API to the cloud. Real public URL. Real logs. Real cost awareness. Tear it down when done to avoid surprise bills.

Note

Kubernetes, SageMaker, and Vertex AI are out of scope for Phase 5. They are great tools for a second job, not a first one. Do not get distracted.

Week 58 — Monitoring, Drift & Retraining

Topics

- Data drift vs concept drift — be able to explain both in 30 seconds
- Prediction logging and storage
- Monitoring dashboards (even a simple Streamlit one counts)
- Alerting on performance degradation
- Retraining strategy: trigger-based vs scheduled

Project

Add monitoring to your deployed API: log every prediction with timestamp and confidence, build a small dashboard showing prediction distribution over time, and implement a simple drift check (e.g., KS-test on input features).

Phase 6 — Capstone + Interview Preparation

Weeks 59–62 • Pull it all together and get the offer

Capstone (Weeks 59–61)

One project. Polished. Deployed. Documented. Better than 90% of junior resumes. Choose a domain you actually care about — the quality of the work rises sharply when you do.

Required components (all of them, no shortcuts)

- Problem framing document — why this matters, who it helps, what success looks like
- Data pipeline with versioning
- Experiment tracking across at least 5 model variants
- Best model selected via proper validation, not by test-set peeking
- FastAPI service with documented endpoints
- Dockerized deployment
- Cloud hosting with a public URL
- Monitoring dashboard or at minimum prediction logging
- Architecture diagram (draw.io, Excalidraw, or similar)
- README that a hiring manager can read in 5 minutes and understand the work
- A short video or Loom walkthrough (optional but high-leverage)

Capstone Domain Suggestions

- Finance: credit risk API with drift monitoring
- Healthcare: risk prediction with calibrated probabilities and honest discussion of limitations
- NLP: domain-specific RAG system with evaluation harness
- CV: defect detection pipeline with Grad-CAM explanations
- Hybrid: recommender system with both classical and LLM components

Interview Preparation (Week 62)

The Three Types of Junior ML Interviews

- **ML fundamentals:** bias–variance, overfitting, metrics, regularization, specific algorithms. Your Phase 2 work covers this.
- **Applied ML / case study:** 'how would you build X?' — problem framing, data, features, model, evaluation, deployment. Your capstone trains this.
- **Coding:** easy–medium LeetCode plus sometimes a SQL round. Your weekly LeetCode habit covers this.

Must-Be-Able-To-Answer Questions

- Explain your capstone end-to-end in 4 minutes, then in 30 seconds
- How would you deploy a model to production? Name every component.

- What is data drift vs concept drift? Give examples of each.
- When would you fine-tune a model vs use RAG vs prompt engineering alone?
- What is LoRA and why does it matter?
- Why does your model do X and not Y? (know your own tradeoffs)
- What would you do differently if you rebuilt your capstone?
- When should you NOT use machine learning?
- Walk me through a time your model failed and what you did.
- How do you evaluate an LLM-based system? What metrics would you use?

Resume, LinkedIn & Portfolio Site

- Projects section first, not education (you are being hired for skills, not credentials)
- Each project: one-line problem, one-line approach, one-line measurable result, tech stack, link
- Quantify where possible — 'reduced error by 18%' beats 'improved accuracy'
- GitHub pinned repos = your portfolio. Pin the 4 best, not the 4 newest.
- LinkedIn: 'Open to work' on, location and role type set, summary written in plain English
- **Portfolio website:** GitHub Pages or a simple Next.js site with project cards linking to repos and live demos. Does not need to be fancy — it needs to exist and load fast.
- **One long-form blog post** on Medium, Dev.to, or your own site walking through your capstone in detail. Hiring managers read these.
- **Short demo video** (2–3 minutes) for your capstone — Loom or a screen recording. Link it from the README and resume.

Behavioral & HR Interviews

Technical skill gets you shortlisted. Behavioral answers get you the offer. Most juniors underprepare here and it shows.

- Prepare the 'tell me about yourself' answer — 90 seconds, story arc, ends with why this role
- STAR format (Situation, Task, Action, Result) for 5–6 stories from your roadmap journey
- Have honest answers for: biggest challenge, biggest mistake, why ML, why this company
- When asked about weaknesses, do not fake humility — pick a real one you are actively working on
- Ask thoughtful questions at the end. 'What does success look like in the first 6 months?' beats 'what's the culture like?'

Mock Interviews

Do at least 5 mock interviews in week 62. Pramp, Interviewing.io, or a study buddy. Record yourself. Watch it back. This is uncomfortable and it works. It is the single highest-leverage thing you can do in the final week.

> ***One final honest note.*** *You will not get offers from every application. Most people need 50–150 applications for their first ML role. That is not a reflection of your skill — it is the math of hiring. Keep building, keep applying, keep shipping. The roadmap ends at week 60, but the career starts there.*

Appendix — Curated Sources

One source per topic. Finish before moving on.

Phase 1

- **Python:** Official Python tutorial + freeCodeCamp Python course (pick one)
- **NumPy:** Official NumPy documentation, 'NumPy: the absolute basics for beginners'
- **Pandas:** Official pandas docs + Kaggle Pandas micro-course
- **SQL:** Mode Analytics SQL Tutorial + StrataScratch for practice
- **Math:** 3Blue1Brown 'Essence of Linear Algebra' and 'Essence of Calculus' (YouTube)

Phase 2

- **Primary:** Andrew Ng's Machine Learning Specialization (Coursera)
- **Book:** Hands-On Machine Learning (Aurélien Géron) — chapters 1–7
- **Library:** scikit-learn official user guide (read, do not just reference)
- **Intuition:** StatQuest with Josh Starmer (YouTube) — for when anything is unclear

Phase 3

- **Primary:** Andrej Karpathy's 'Neural Networks: Zero to Hero' (YouTube) — this is the gold standard
- **Framework:** PyTorch official tutorials
- **CV:** Stanford CS231n notes (skim, not the full course)
- **NLP:** Hugging Face NLP course
- **Transformers:** 'The Illustrated Transformer' by Jay Alammar

Phase 4 (LLM Apps + Fine-Tuning)

- **Fundamentals:** Anthropic and OpenAI API docs (read them directly, not a course about them)
- **RAG:** LlamaIndex or LangChain docs — pick one, do not learn both at first
- **Evaluation:** Hamel Husain's 'Your AI Product Needs Evals' blog post
- **Vector DBs:** Chroma docs to start; explore Qdrant/Pinecone later
- **Fine-Tuning:** Hugging Face PEFT documentation + Sebastian Raschka's 'Finetuning LLMs' blog series
- **QLoRA hands-on:** 'Fine-Tune Your Own Llama 2 Model' (Maxime Labonne, Towards Data Science) — adapt for Llama 3 or Mistral
- **Conceptual:** Chip Huyen's 'Building LLM Applications for Production' blog post — best overview of when to fine-tune vs RAG vs prompt

Phase 5 (MLOps + Cloud)

- **MLOps overview:** 'Machine Learning Engineering for Production' (Coursera, DeepLearning.AI)
- **FastAPI:** Official FastAPI docs — genuinely excellent, better than most tutorials
- **Docker:** 'Docker Curriculum' (docker-curriculum.com)
- **AWS:** AWS 'Cloud Practitioner Essentials' free course + hands-on
- **Monitoring:** Evidently AI blog posts on drift detection

Communities Worth Joining

- MLOps Community (Slack)
- Hugging Face Discord
- r/MachineLearning and r/LearnMachineLearning on Reddit (filter carefully)
- Local AI/ML meetups — in-person connections beat online ones

Final Thought

A roadmap is a map, not a track. You will deviate. You will get stuck on things not in the plan. You will find topics you love that deserve extra time. That is the work. What matters is not following this document exactly — it is building real things, explaining them honestly, and shipping them publicly for 14 months straight.

If you do that, the offer follows. If you do not, no roadmap will save you. Good luck.